Prueba Objevtiva RA2: gestión de una biblioteca con JDBC y SQLite

Módulo: Acceso a Datos (2.º DAM)
Tecnologías: Java 21, Maven, JDBC, SQLite y JUnit 5.
Objetivo: implementar **tres DAO incrementales**, siguiendo la estructura del ejercicio de clínica veterinaria. El código de los DAO se entrega sin implementar; los modelos, contratos, esquema de base de datos y pruebas están preparados.

1. Contexto

Una biblioteca quiere almacenar sus libros, sus socios y los préstamos realizados. Debes desarrollar la capa de acceso a datos con JDBC y SQLite. No es necesario construir interfaz gráfica ni aplicación de consola.

Cada nuevo DAO amplía el nivel de dificultad del anterior:

1. `LibroDao`: CRUD sobre una tabla independiente.
2. `SocioDao`: CRUD y dos consultas de búsqueda filtrada.
3. `PrestamoDao`: CRUD con relaciones entre entidades y dos métodos adicionales de consulta.

2. Esquema de base de datos

Las tablas y sus restricciones están definidas en `src/main/resources/schema.sql`.

- `libros(id, titulo, autor, anio_publicacion)`: `titulo` y `autor` obligatorios.
- `socios(id, nombre, email, fecha_alta)`: `email` obligatorio y único.
- `prestamos(id, id_libro, id_socio, fecha_prestamo, fecha_devolucion)`: claves foráneas obligatorias a `libros` y `socios`; `fecha_devolucion` puede ser nula.

Las fechas se representan como texto ISO `YYYY-MM-DD`. Un libro o socio con préstamos asociados no puede eliminarse: debes respetar la integridad referencial de los datos. Una misma obra puede registrarse en múltiples préstamos históricos (no se pide bloquear préstamos simultáneos).

> **Pista:** para que las relaciones y restricciones entre tablas funcionen correctamente con SQLite, es necesario configurar algo al abrir cada conexión desde Java. Revisa la documentación del controlador JDBC y de SQLite. Las pruebas usan una base temporal independiente por test.

3. DAO 1: LibroDao (nivel básico, 2 puntos)

Implementa `ILibroDao` usando `LibroDao`:

Método	Contrato
`int insertar(Libro entidad)`	Inserta y devuelve el ID positivo generado.
`Optional<Libro> buscarPorId(Integer id)`	Devuelve el libro o `Optional.empty()`, también si `id` es nulo.

Método	Contrato
`List<Libro> listar()`	Devuelve todos los libros, ordenados por ID ascendente.
`int actualizar(Libro entidad)`	Modifica todas las columnas editables y devuelve las filas afectadas.
`int eliminar(Integer id)`	Elimina por ID y devuelve las filas afectadas.

La inserción no debe aceptar títulos nulos. `actualizar` sin ID y `eliminar` con ID nulo deben generar `IllegalArgumentException`. El resto de errores de acceso a datos podrán encapsularse en `DaoException`.

4. DAO 2: SocioDao (nivel intermedio, 3 puntos)

Implementa las mismas cinco operaciones CRUD definidas en `ISocioDao`. `listar()` se ordenará por ID ascendente. A continuación, añade estas **dos consultas nuevas**:

Consulta 1 — Búsqueda por correo electrónico exacto

Implementa la consulta SQL correspondiente utilizando parámetros.

Firma: `Optional<Socio> buscarPorEmail(String email)`. Cuando no hay coincidencia, devuelve `Optional.empty()`.

Consulta 2 — Altas desde una fecha (inclusive)

Implementa la consulta SQL correspondiente; incluye la fecha indicada y ordena por fecha e ID.

Firma: `List<Socio> listarAltasDesde(String fecha)`. Devuelve lista vacía si no hay coincidencias. Se espera formato ISO `YYYY-MM-DD`.

Las consultas deberán utilizar parámetros mediante `PreparedStatement`. El correo no puede repetirse.

5. DAO 3: PrestamoDao (nivel avanzado, 4 puntos)

Implementa el CRUD de `IPrestamoDao`. Los préstamos deben referenciar un libro y un socio existentes; en `listar()` los resultados estarán ordenados por ID. `actualizar()` modificará las columnas editables de un préstamo, incluida su fecha de devolución.

Consulta 3 — Número de préstamos por socio

Obtener el ID y nombre de cada socio y su número total de préstamos, **incluyendo socios con cero préstamos**. Ordenar primero por total descendente y después por ID ascendente.

Resuelve esta consulta sobre las entidades relacionadas.

Firma: `List<PrestamosPorSocio> contarPrestamosPorSocio()`. El DTO `PrestamosPorSocio` está incluido.

Consulta 4 — Préstamos de libros de un autor

Devuelve los préstamos de los libros cuyo autor coincide exactamente con el indicado, ordenados por ID ascendente.

Firma: `List<Prestamo> listarPrestamosPorAutor(String autor)`. Si no hay coincidencia, devolverá una lista vacía.

6. Requisitos de implementación

- Implementa **solo** las clases `LibroDao`, `SocioDao` y `PrestamoDao`. Sus interfaces y modelos están proporcionados.
- No modifiques las firmas de los métodos ni las pruebas JUnit.
- Utiliza JDBC (`Connection`, `PreparedStatement`, `ResultSet`) y cierra los recursos con `try-with-resources`.
- Respeta la integridad referencial y las restricciones SQL.
- Usa `Optional` y `List` según los tipos de retorno definidos.
- Evita concatenar valores de usuario en las sentencias SQL.
- **Documenta los métodos de la API en el lugar correcto**, empleando JavaDoc con una descripción precisa, las etiquetas `@param` y `@return` cuando correspondan y palabras clave vinculadas a la finalidad de cada método. Consulta las interfaces proporcionadas y conserva las firmas originales.

7. Pruebas automáticas

La organización mantiene el modelo del ejercicio veterinario: una clase base y una clase de test por DAO, **16 test por DAO y 48 en total**:

Clase	Tests	Comprueba
`LibroDaoTest`	16	CRUD básico, nulos, orden, fallos e interfaz.
`SocioDaoTest`	16	CRUD, unicidad de correo y dos consultas parametrizadas.
`PrestamoDaoTest`	16	CRUD, integridad referencial y dos métodos de consulta adicionales.

`BaseDaoTest` prepara y elimina el fichero SQLite temporal en cada prueba; **no** hay `IntegracionDaoTest` independiente.

La descripción JavaDoc de cada test explica exactamente el comportamiento esperado. Cada prueba se identifica mediante un método `@Test` terminado en `Test`.

Para ejecutar únicamente las pruebas:

```
mvn clean test
```

8. Cómo consultar la nota

Ejecuta **desde la raíz del proyecto**, donde está `pom.xml`:

```
mvn clean verify -Pnota
```

Este comando compila, ejecuta las pruebas JUnit y, durante la fase `verify`, llama a `calcular_nota.py` mediante el perfil Maven `nota`. El informe aparece por pantalla y se guarda en:

```
target/notas_por_bloques.txt
```

Requisitos del cálculo: Maven, JDK 21 y `python3` accesible desde el terminal. El perfil `nota` no detiene la generación del informe por fallos de los test; sin embargo, si el código **no compila**, Maven no llegará a generar la nota. Corrige los errores de compilación antes de calcularla.

Rúbrica de evaluación ordinaria (sobre 10 puntos)

La puntuación de cada DAO se desglosa en su **CRUD básico** y, cuando corresponda, sus **métodos adicionales**. La rúbrica no identifica las técnicas SQL que deben usarse: los requisitos funcionales y el resultado esperado de cada método se especifican en el enunciado y las interfaces.

DAO	CRUD básico	Métodos adicionales	Total del DAO
`LibroDao`	2,00	—	**2,00**
`SocioDao`	2,00	1,50 (dos métodos)	**3,50**
`PrestamoDao`	2,00	1,90 (dos métodos)	**3,90**
Subtotal funcionalidad	**6,00**	**3,40**	**9,40**
Documentación JavaDoc			**0,60**
TOTAL			**10,00**

Los 1,50 puntos adicionales de `SocioDao` se distribuyen entre las pruebas de sus dos métodos nuevos. Los 1,90 puntos adicionales de `PrestamoDao` corresponden a 0,95 puntos por cada uno de sus dos métodos nuevos.

Condición de la documentación: los 0,60 puntos se añaden únicamente cuando se superan **todos** los test del CRUD de **los tres DAO**. La documentación de la API debe ser correcta: descripción (40 %), etiquetas `@param` (25 %), etiqueta `@return` (20 %) y palabras clave relacionadas con la finalidad del método (15 %). No se puntúa `@throws`.

Cada apartado funcional se puntúa proporcionalmente al número de pruebas superadas de su grupo. Completar los tres CRUD da hasta 6,00 puntos y, si la documentación es correcta, se alcanzan 6,60 puntos sin necesidad de superar las consultas adicionales. La nota ordinaria máxima es 10 puntos.

Mejoras voluntarias: hasta 2 puntos adicionales

Se valorará la calidad del diseño y de la implementación mediante **hasta 2,00 puntos extra**, independientemente de que se hayan completado los métodos adicionales de los DAO. Por tanto,

también puede obtenerse esta puntuación extra si solo se desarrollan los CRUD básicos. No basta con mencionar las mejoras: deben estar aplicadas y poder comprobarse en el código entregado.

Mejora evaluable	Máximo	Evidencias esperadas
Verificación y robustez	**0,75**	Incorporación de pruebas propias adicionales, tratamiento coherente de parámetros y casos límite, y comprobación de restricciones sin modificar los test proporcionados.
Abstracción y reutilización	**0,75**	Extracción de responsabilidades compartidas (por ejemplo, apertura y configuración de conexiones), eliminación de duplicidades y separación de responsabilidades sin cambiar los contratos de las interfaces.
Optimización y calidad de acceso a datos	**0,50**	Gestión eficiente de conexiones y recursos, consultas ajustadas a los datos necesarios y evitación de operaciones redundantes; las mejoras deben estar justificadas.
Total de mejoras	**2,00**	

Cómo se calcula la nota: `mvn clean verify -Pnota` seguirá mostrando la **nota ordinaria automática** (0–10), basada en los 48 test y el JavaDoc. El profesorado revisará las mejoras voluntarias y asignará entre 0 y 2 puntos adicionales, según las evidencias aportadas. La **nota final** será `min(10, nota ordinaria + puntos extra)`. Por ejemplo, quien obtenga 6,60 por los CRUD y la documentación podrá alcanzar **8,60** con todas las mejoras, aunque no implemente los métodos adicionales. La puntuación extra no sustituye la obligación de entregar un proyecto que compile y pueda revisarse.

Las mejoras deben ser compatibles con las interfaces, las firmas y los test proporcionados. Añade una breve sección **«Mejoras realizadas»** en la memoria de entrega o en un archivo `MEJORAS.md`, indicando qué has implementado, dónde se encuentra y cómo se verifica. **No modifiques el README del enunciado ni las pruebas facilitadas.**

9. Entrega

Entrega el proyecto Maven completo con los tres DAO implementados y ejecuta `mvn clean verify -Pnota` antes de entregarlo. No alteres las pruebas ni el baremo automático. Si optas a la puntuación extra, adjunta `MEJORAS.md` con las evidencias de cada mejora.